



中国科学技术大学

模块化程序设计

徐 伟

E-mail: xuweihf@ustc.edu.cn



课程概述

- ❖ 模块化编程概念与原则
- ❖ 头文件设计与使用规范
- ❖ 多文件编译与链接



模块化编程概念与原则

❖ 模块

∞ 硬件模块

- ❖ 将计算机的某一种或几种特定功能设计成一个独立的、可以插拔的硬件部件

∞ 内存、硬盘、CPU、显卡...

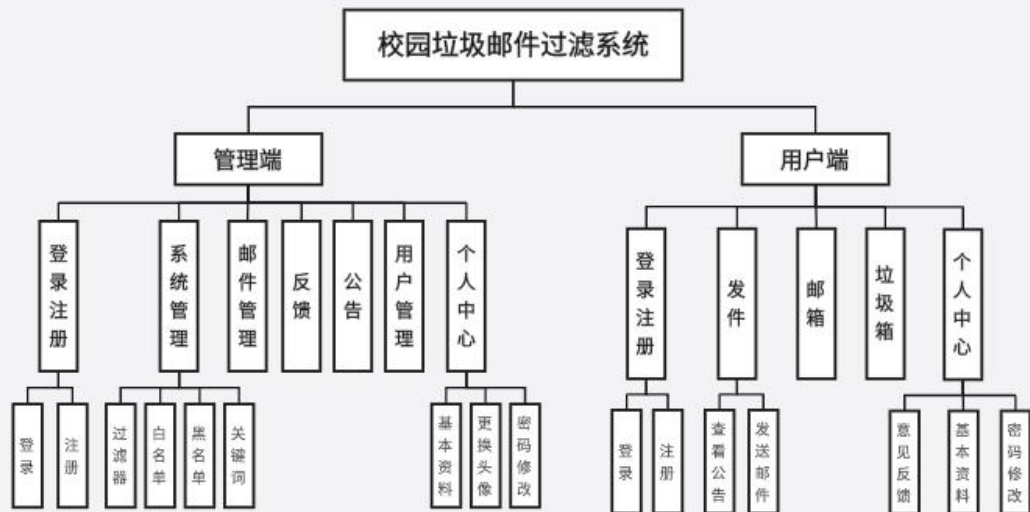
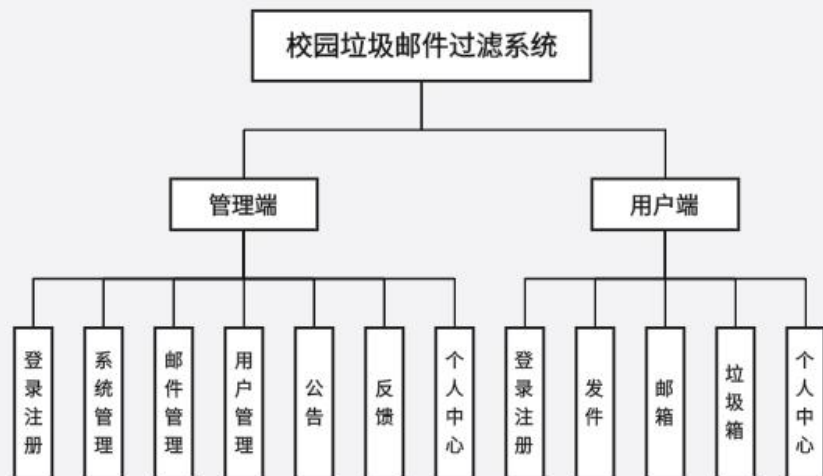
∞ 软件模块

- ❖ 将复杂的软件系统，根据功能，拆分成一个个独立的、可以互相协作的代码块（模块）
- ❖ 每个模块负责一项具体的任务，对外隐藏了内部的实现细节，只留下一些“接口”供其他模块使用

∞ 函数、库...



模块图

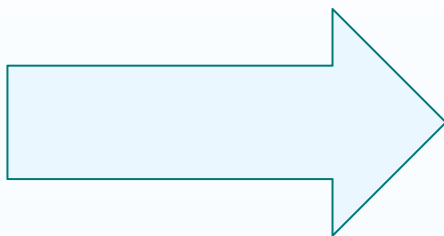




单文件与多文件开发

❖ 单文件

- ❧ 代码全在一个文件
- ❧ 快速原型验证
- ❧ 个人项目足够



❖ 多文件

- ❧ 代码复用
- ❧ 独立测试
- ❧ 迭代演进
- ❧ 避免冲突

核心问题:

如何把一个大程序拆成多个小模块
使其易维护、可复用、可测试？



模块化设计三原则

❖ 高内聚 (High Cohesion)

- ∞ 一个模块功能单一、职责明确，内部各成分（语句、函数）之间的关联程度要高，共同完成一个明确的功能。

❖ 低耦合 (Low Coupling)

- ∞ 模块与模块之间的相互依赖程度要低，接口简单清晰。
- ∞ 修改一个模块，应尽量不影响其他模块。

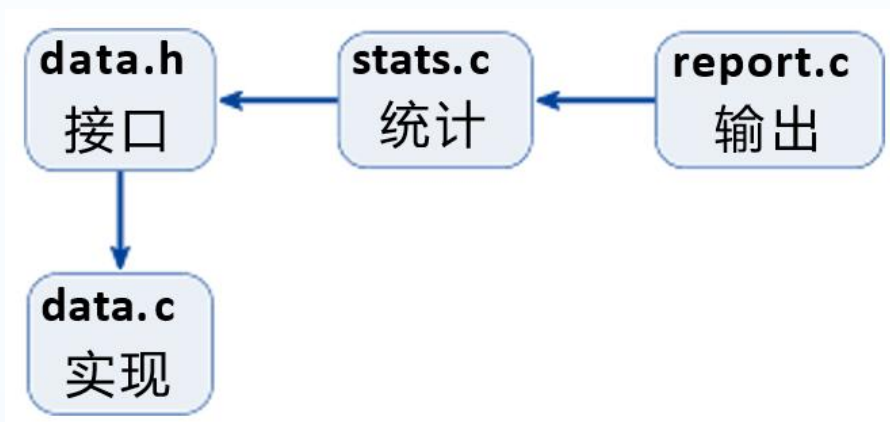
❖ 信息隐藏 (Information Hiding)

- ∞ 模块内部的具体实现细节应该被隐藏起来，只通过一个明确的接口（API）对外提供服务。
- ∞ 使用者只需知道“怎么用”，无需知道“怎么实现”。



模块化设计三原则

例：成绩统计程序



高内聚： stats.c只做统计，data.c只负责读取

低耦合： stats.c只依赖data.h 接口（函数调用），不关心文件格式

信息隐藏： data.h只声明struct Data 和函数接口



模块边界

- ❖ 模块的边界是指模块与模块之间的界限，决定了模块的独立性和可复用性
- ❖ 模块的边界通过接口和依赖关系来定义，接口定义了模块对外提供的功能，依赖关系定义了模块与其他模块之间的关系



模块边界如何界定？

一、按功能界定（最核心）：

只做一件事。例如：输入模块、计算模块、验证模块、打印模块；

二、按业务逻辑边界界定：

根据问题本身的逻辑自然拆分。如学生管理系统：录入模块、查询模块、修改模块、删除模块、统计模块

三、按复用性界定：

多次重复使用的代码，独立成模块；公共功能（如登录、注册）抽成公用模块

四、按复杂度界定：

模块代码量适中，不能太大，也不能太小



模块化设计

❖ 不好的模块划分

- ❧ 不能用一句话描述模块的功能（功能不单一）
- ❧ 一个模块改动影响其他很多模块（耦合度高）
- ❧ 模块之间交叉调用、循环调用（耦合度高）
- ❧ 多个模块共用全局变量（耦合度高）





写 .h 之前先做接口设计

❖ 接口设计的几个原则

- 单一职责原则：让接口专注做一件事，用一句话描述功能
- 定义清晰的输入输出规范：函数参数、返回值
- 最小暴露原则：隐藏实现细节，降低耦合度
- 最少依赖原则：
去掉这个模块，其他模块还能不能正常运行？



问题1 模块能力

❖ 这个模块提供什么能力？（最小集合）

∞ 含义：明确模块的核心功能，避免“什么都做”

∞ 例子：vector 模块只提供：创建、销毁、添加元素、获取元素、查询大小

∞ 反例：在 vector 模块里加排序、搜索等功能

❖ 这些应该是独立模块



问题2输入输出

❖ 输入/输出是什么？返回值是否足够表达错误？

∞ 含义：函数参数和返回值要能表达所有可能的情况

∞ 好例子： `int vec_push(Vec* v, int x)` 返回0成功， -1失败

∞ 坏例子： `void vec_push(Vec* v, int x)` 无法知道是否成功



问题3 隐藏实现

❖ 如何隐藏实现（利用不透明指针）？

❧ 方法：利用不透明指针，让使用者看不到结构体的内部细节

❧ 好处：隐藏实现细节，修改内部字段不影响使用者的代码

❧ 例子：

❖ `typedef struct Vec Vec;` ——只暴露类型名（句柄）

❖ 结构体定义放在 `.c` 文件里，使用方看不到



问题3 隐藏实现

❖ 什么是句柄 (Handle)

- ☞ 用于标识和访问某个资源的间接引用，通常是一个无符号长整数，它通过映射表指向实际的内核对象地址，外部代码只能持有这个值，而无法直接“看到”或操作它所指向的内部数据结构。

```
#include <stdio.h>
```

//FILE* 就是一个典型的句柄。

```
int main() {  
    // 获取文件句柄  
    FILE* fileHandle = fopen("example.txt", "r");  
    if (fileHandle == NULL) {  
        // 错误处理  
        return 1;  
    }  
    ...  
}
```



问题3 隐藏实现

暴露结构体

```
// vec.h
struct Vec {
    int* data;
    int size;
    int capacity;
};
```

问题

- ▶ 调用方可以直接访问 `v->size`
- ▶ 修改字段触发大量重编译
- ▶ 难以改变内部实现

不透明指针 (核心: 前置声明)

```
// vec.h
typedef struct Vec Vec;

size_t vec_size(const Vec* v);

// vec.c
struct Vec {
    int* data;
    int size;
    int capacity;
};
```

好处

- ▶ 调用方只能通过 `vec_size()` 使用
- ▶ 修改内部不影响 `.h`
- ▶ 可以随时优化实现



前置声明 (Forward Declaration)

❖ 前置声明 (Forward Declaration, 又译作“前向声明”)

是C/C++中的一种技术, 用于在声明某个实体 (通常是类、函数、变量等) 的名称而无需提供其详细定义。目的是为了告诉编译器某个实体的存在, 以便在稍后的代码中引用它, 而不必在声明的地方提供完整的定义。这可以提高编译速度和减少编译依赖性



前置声明

❖ 为什么需要前置声明

∞ 解决相互依赖

❖ 当两个模块互相引用时，必须有前置声明才能编译通过

```
//a.h
#include "b.h"
class B;
class A
{
...
private:
B b;
};
```

```
//b.h
#include "a.h"
class A;
class B
{
...
Private:
A a;
};
```



前置声明

❖ 为什么需要前置声明（续）

∞ 隐藏实现

❖ .h头文件中，只给出类型的前置声明，定义放源文件（.c）

// 在头文件 widget.h 中

struct Widget; // 结构体前置声明: **Widget** 是一个结构体，但不知道成员细节

(建立存储空间的声明称之为“定义”，而把不需要建立存储空间的称之为“声明”。)

// 使用指针操作，因为指针大小已知

void widget_process1(struct Widget* w); // 函数的前置声明

void widget_process2(struct Widget w); // 错误，因为**Widget**定义未知

// widget.c

struct Widget {

int id;

char name[32];

// ... 其他成员

};



前置声明

❖ 为什么需要前置声明（续）

∞ 减少编译依赖

- ❖ 头文件只需要知道某个类型存在（比如作为指针参数），不需要知道其大小或成员
- ❖ 用前置声明代替包含完整的定义，避免不必要的重新编译

//main.c

```
#include "widget.h"
```

.....

widget.c中修改结构体的定义，不会导致main.c重新编译



前置声明的限制

❖ 当编译器需要知道结构体大小时，必须有完整的定义

可以使用前向声明

```
// 只用指针
typedef struct Vec Vec;
void process(Vec* v);

struct Data {
    Vec* vec;
};
```

指针大小固定（通常 8 字节），
编译器不需要知道 **Vec** 的完整结构

必须include完整定义

```
// 需要知道大小
void process(Vec v); // 错误!
struct Data {
    Vec vec; // 错误!
};

int size = sizeof(Vec); // 错误!
v->data[0] = 1; // 错误!
```

编译器需要知道 **Vec** 的大小和字段布局



课程概述

- ❖ 模块化编程概念与原则
- ❖ 头文件设计与使用规范
- ❖ 多文件编译与链接



思考题

- ❖ 如果把所有结构体细节都放到 `.h`, 会发生什么?



思考题

❖ 如果把所有结构体细节都放到.h, 会发生什么?

❧ 封装性破坏

❧ 编译依赖爆炸, 构建变慢

❧ 兼容性丧失

❧ 安全风险

❧ ...



为什么修改头文件结构体导致重新编译？

场景：结构体定义在头文件中

```
// vec.h (暴露了结构体细节)
struct Vec {
    int* data;
    int size;           // 假设我们要修改这个字段
    int capacity;
};
```

有 3 个文件包含了这个头文件

```
// a.c b.c c, c都包含了 vec.h
#include "vec.h"
```

当修改结构体字段时（比如把 `int size` 改成 `size_t size`）会如何？



为什么修改头文件结构体导致重新编译？

❖ 原因分析

⌘ 预处理机制

- ❖ #include是文本替换，头文件内容被“复制”到每个包含它的 .c 文件中

⌘ 依赖传播

- ❖ 如果 vec.h 被 a.c, b.c, c.c 包含，修改 vec.h 会导致这 3 个文件都需要重新编译

⌘ 时间戳检查

- ❖ Make等构建工具会检查文件修改时间，发现 .h 更新后会重新编译所有依赖它的.c



对比：使用不透明指针

改进方案：使用不透明指针（Opaque Pointer）

```
// vec.h (只暴露类型名)
typedef struct Vec Vec;      //前向声明, 不暴露细节
Vec* vec_create(void);
void vec_destroy(Vec* v);
```

```
// vec.c (结构体定义放在这里)
struct Vec {
    int* data;
    size_t size;  // 修改这里不影响 vec.h
    size_t capacity; };
```

好处：修改 `vec.c` 中的结构体字段，`vec.h` 不变



如何减少重新编译

❖ 使用前向声明 + 不透明指针

☞ 减少头文件中的include

❖ 信息隐藏

☞ 实现细节放到.c，只在.h中暴露必要的接口

❖ 合理组织模块

☞ 避免大而全的头文件



头文件模板

```
// vec.h (公共接口)
#ifndef VEC_H
#define VEC_H

#include <stddef.h>    // 只include自己真的需要的

typedef struct Vec Vec; // 不透明指针: 隐藏实现细节

Vec* vec_create(void);
void vec_destroy(Vec* v);
int  vec_push(Vec* v, int x); // 返回错误码
size_t vec_size(const Vec* v);

#endif // VEC_H
```

#ifndef/#define: 包含保护, 防止头文件被重复包含导致重复定义



头文件设计注意事项

- ❖ 包含保护

 - ☞ #ifndef/#define/#endif 或 #pragma once

- ❖ 自治

- ❖ 依赖最小化

- ❖ 禁止循环包含

- ❖ 避免变量定义

- ❖ 避免函数实现



自洽

- ❖ 一个头文件是自洽（Self-Contained）的，说明：
 - ☞ 它独自被#include时能够编译通过，不依赖其他头文件被提前包含
 - ☞ 它显式地包含了直接使用的所有类型、宏、函数的声明



自洽

- ❖ 一个头文件是自洽（Self-Contained）的，说明：
 - ☞ 它独自被#include时能够编译通过，不依赖其他头文件被提前包含
 - ☞ 它显式地包含了直接使用的所有类型、宏、函数的声明

反例：隐式依赖

```
// vec.h
```

```
void vec_add(Vector* v, size_t n); // 使用了 size_t, 但没有包含 <stddef.h>
```

若某个.c文件引用了vec.h

```
#include "vec.h"
```

```
// 编译错误! size_t 未定义
```




自洽

- ❖ 一个头文件是自洽（Self-Contained）的，说明：
 - ☞ 它独自被#include时能够编译通过，不依赖其他头文件被提前包含
 - ☞ 它显式地包含了直接使用的所有类型、宏、函数的声明

必须改为

```
#include <stddef.h>
```

```
#include "vec.h" // 正确，但依赖包含顺序
```

“必须先包含 A 才能包含 B”的依赖就是不自洽



自洽

❖ 自洽（Self-Contained）重要性

∞ 避免隐式顺序约束

❖ 使用者无需记住头文件的包含顺序，降低出错概率。

∞ 提高可维护性

❖ 修改头文件时，不会因为依赖关系隐藏而破坏现有代码。

∞ 简化代码审查和重构

❖ 每个头文件都是独立的单元，易于理解和测试



自洽

- ❖ 一个头文件是自洽（Self-Contained）的，说明：
 - ❧ 它独自被#include时能够编译通过，不依赖其他头文件被提前包含
 - ❧ 它显式地包含了直接使用的所有类型、宏、函数的声明

自洽性测试

写一个简单的测试文件 `test_vec.c`:

```
#include "vec.h" // 仅包含这一行
```

如果能编译通过，说明头文件是自洽的

自洽的头文件 = 独立、可靠、易用的接口



依赖最小化

- ❖ 采用前向声明等方式，减少依赖
 - ∞ 只声明类型名，不包含完整定义
 - ∞ 减少依赖，只用指针时不需要 `#include`

```
// a.h
```

```
typedef struct A A;      // 前向声明
```

```
typedef struct B B;      // 前向声明
```

```
void a_do_something(A* a, B* b); // 只使用指针，不需要完整定义
```



禁止循环包含

❖ 循环包含

- ❧ 头文件 A 直接或间接地包含了头文件 B，而头文件 B 又直接或间接地包含了头文件 A

```
// a.h  
#include "b.h"
```

```
// b.h  
#include "a.h"
```

❖ 循环包含的问题

- ❧ 编译错误
- ❧ 设计耦合
- ❧ 难以维护



为什么头文件保护（#ifndef）不能解决循环包含？

- ❖ 头文件保护可以防止同一个头文件被**重复包含**，但无法打破循环依赖的逻辑

```
// a.h
#ifndef A_H
#define A_H
#include "b.h"
typedef struct A { B* b; } A;
#endif
```

当编译 **a.c** 包含 **a.h** 时，会发生什么？

```
// b.h
#ifndef B_H
#define B_H
#include "a.h"
typedef struct B { A* a; } B;
#endif
```



如何避免循环包含

- ❖ 使用前置声明（Forward Declaration）代替包含
- ❖ 将互相依赖的类型放到同一个头文件中
- ❖ 检查循环包含的工具
 - ❧ 头文件依赖检查软件（如include-graph等）
- ❖ 使用`#pragma once`
 - ❧ 不能解决设计上的循环
 - ❧ 可以防止同一文件多次包含，配合前向声明使用更简洁



#pragma once工作原理

- ❖ 预处理器处理到#pragma once时，它会记录当前文件的唯一标识。当后续再次遇到#include指向同一个文件时，预处理器会检查该文件是否已经在列表中：
 - ❧ 如果已经在列表中，则完全跳过该文件的内容
 - ❧ 如果不在列表，则正常展开文件内容，并将其加入列表
- ❖ 不依赖宏定义
 - ❧ 传统的头文件保护（#ifndef）需要手动定义一个宏，并检查该宏是否存在
 - ❧ #pragma once 是编译器直接提供的机制，不需要程序员操心宏名的唯一性



#pragma once工作原理

```
// example.h
#pragma once // 只出现一次，无需额外宏

struct Example {
    int value;
};
```



避免变量定义

❖ 除非static const或extern声明

- ❧ static变量具有内部链接，即只在定义它的编译单元（当前 .c 文件及其直接或间接包含的头文件）内可见
- ❧ extern声明变量而不定义，告诉编译器变量在别处定义

```
// counter.h
```

```
extern int g_count;    // 只是声明，不分配内存
```

```
// counter.c
```

```
int g_count = 0;      // 真正的定义，分配内存
```



避免函数实现

- ❖ 当头文件被多个 .c 文件包含时，导致多重定义
 - 🌀 链接时，链接器发现多个相同的符号，会报“**重复定义**”错误
- ❖ 编译时间增加
- ❖ 代码膨胀



避免函数实现

❖ 场景：util.h 中定义了函数 add，两个 .c 文件都包含了它

文件结构

```
# util.h
int add(int a, int b) {
    return a + b;
}
```

```
# main.c
#include "util.h"
int main() { return add(1,2); }
```

```
# test.c
#include "util.h"
void test () { add(3,4); }
```

编译过程

```
# 编译 main.c
gcc -c main.c → main.o
main.o 包含 add 的定义
# 编译 test.c
gcc -c test .c → test .o
test .o 也包含 add 的定义
# 链接
```

```
gcc main.o test .o -o app
错误！链接器发现 add 被定义了2
次 → multiple definition of 'add '
```



模块合理命名

- ❖ 避免不同模块的函数名冲突（C语言没有命名空间）
- ❖ 同一模块的所有函数用统一前缀（如 `vec_*`、`logger_*`）

例：

```
vec_create()  
vec_push()  
vec_size()
```



头文件常见警告和错误

- ❖ Error: 编译/链接失败, 无法生成可执行文件
- ❖ Warning: 可能有问题, 但仍能编译成功

Error (编译/链接期错误)

► multiple definition of ...

阶段: 链接期

结果: 无法生成可执行文件

► undefined reference to ...

阶段: 链接期

结果: 找不到某个符号的具体定义, 无法生成可执行文件

► unknown type name 'X'

阶段: 编译期

结果: 从未见过 X 的定义或声明, 无法生成.o

Warning (警告)

► unused variable

阶段: 编译期

结果: 可以编译, 但提示有未使用的变量

► format specifies type 'int' but the argument has type 'long'

阶段: 编译期

结果: 可以编译, 但提示有类型不匹配的变量



头文件常见警告和错误

multiple definition of ...

同名定义出现在多个翻译单元

常见原因：在 .h 写了变量/函数定义

undefined reference to ...

声明存在但实现没参与链接

常见原因：漏编译某个 .c / 库链接顺序不对

include 循环

A 包含 B , B 又包含 A

解决方法：用前向声明 / 抽公共层



头文件常见警告和错误

预处理: **#include** 是文本替换

► 头文件内容会被“复制粘贴”进每个 .c

► 所以: 循环包含、宏污染、头文件变“胖”都会产生连锁效应

编译: 每个 .c 都是独立翻译单元

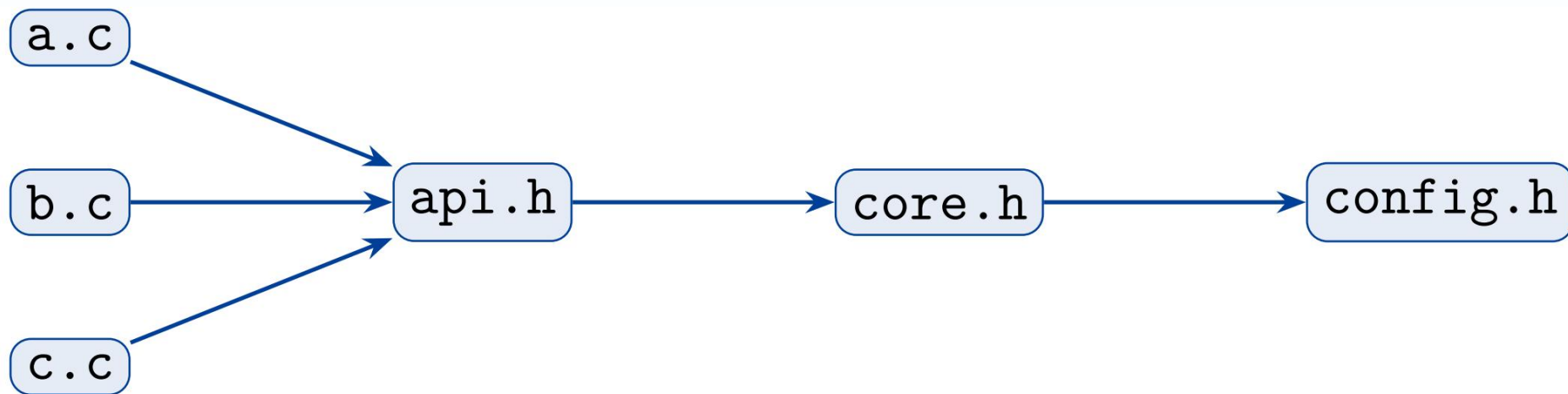
► 每个 .c 单独编译成 .o

► 如果头文件里放了“定义”, 每个 .o 都会各自带一份

► 链接时就会出现 **multiple definition**



重编译风暴的消除



- ❖ #include 是文本替换，会带来传递依赖
- ❖ 头文件越“胖”，重编译范围越大
- ❖ 头文件规范（前向声明/最小包含）能显著减少耦合



重编译风暴的消除

❖ **场景：**修改了 `config.h`

依赖链条

文件包含关系

`a.c` includes `api.h`

`api.h` includes `core.h`

`core.h` includes `config.h`

传递依赖

`a.c` → `api.h` → `core.h` → `config.h`

解释：

- ▶ `a.c` 包含 `api.h`
- ▶ `api.h` 包含 `core.h`
- ▶ `core.h` 包含 `config.h`
- ▶ 所以 `a.c` 间接依赖 `config.h`

重编译过程

修改 `config.h` 后

1. `config.h` 时间戳更新
2. `core.h` 依赖 `config.h` → 重编译
3. `api.h` 依赖 `core.h` → 重编译
4. `a.c`, `b.c`, `c.c` 依赖 `api.h` → 全部重编译!

结果：

- ▶ 修改 1 个文件 (`config.h`)
- ▶ 触发 3 个 `.c` 文件重编译
- ▶ 这就是 ” 重编译风暴 ” !



重编译风暴的消除

❖ 核心思想：减少头文件依赖，使用前向声明

不好的做法

```
// api.h (暴露太多)
#include "core.h" // 完整包含
#include "config.h"
#include "logger.h"
#include "utils.h"
typedef struct Data {
    CoreType core; // 需要完整定义
} Data;
```

问题：

► 包含了 4 个头文件

好的做法

```
// api.h (最小包含)
#include <stddef.h> // 只包含必需的

// 前向声明，不包含头文件
typedef struct CoreType CoreType;
typedef struct Data {
    CoreType* core; // 只用指针
} Data;
```

好处：

► 只包含 1 个标准库头文件



课程概述

- ❖ 模块化编程概念与原则
- ❖ 头文件设计与使用规范
- ❖ 多文件编译与链接



什么是makefile

❖ Small programs → single file

❖ 项目规模不断增大

- ❧ Many lines of code
- ❧ Multiple components
- ❧ More than one programmer

❖ 项目规模增大，带来的问题

- ❧ 长文件难于管理
- ❧ 每次修改均需要长时间编译
- ❧ 多开发人员无法同时修改同一文件



什么是makefile

❖ 解决办法

- ❧ 将工程拆分到多个源文件

❖ 目标

- ❧ 对模块进行良好切分
- ❧ 缩短编译时间，仅对修改的部分进行重新编译
- ❧ 易于维护项目结构
 - ❖ 依赖和创建



什么是makefile

❖ 办法采用makefile

- ∞ 使用make命令和makefile工具
- ∞ 理顺各个源文件之间纷繁复杂的相互关系



makefile安装

❖ Centos

❧ `sudo yum -y install gcc automake autoconf libtool make`

❧ `sudo yum install gcc gcc-c++`

❖ Ubuntu

❧ `sudo apt-get update`

❧ `sudo apt-get install make`



Makefile基础

❖ 显示规则

- ❧ 目标文件：依赖文件
- ❧ 原则上第一个文件就是最终的文件
- ❧ 缩进必须TAB键（必须的，不可省略）

```
1 test:test.o      #目标文件test, 依赖文件test.o
2     gcc test.o -o test
3 test.o:test.s
4     gcc -c test.s -o test.o
5 test.s:test.i
6     gcc -S test.i -o test.s
7 test.i:test.c
8     gcc -E test.c -o test.i
```

假设单文件 test.c 编译

采用递归方式书写

使用make命令自动编译出
目标文件test

#表示注释



Makefile基础

❖ 变量

- ❧ 变量一般定义在文件头部
- ❧ 使用\$(变量名)表示变量

符号	含义
=	赋值
+=	增加赋值
:=	常量赋值

```
1  TAR = test          #赋值
2  CC := gcc           #常量赋值
3
4  $(TAR):test.o        #常量使用
5      $(CC) test.o -o $(TAR)
6  test.o:test.s
7      $(CC) -c test.s -o test.o
8  test.s:test.i
9      $(CC) -S test.i -o test.s
10 test.i:test.c
11     $(CC) -E test.c -o test.i
```



Makefile基础

❖ 多文件编译

示例circle.c circle.h cube.c cube.h test.c编译

```
├── circle.c  
├── circle.h  
├── cube.c  
├── cube.h  
├── makefile  
└── test.c
```



Makefile基础

❖ 多文件编译

示例circle.c circle.h cube.c cube.h test.c编译

```
#include <stdio.h>

int circle();
```

circle.h

```
#include <stdio.h>

int cube();
```

cube.h

```
#include "circle.h"

int circle()
{
    printf("This is cicle fun\n");
    return 0;
}
```

circle.c

```
#include "cube.h"

int cube()
{
    printf("This is cube fun\n");
    return 0;
}
```

cube.c



Makefile基础

❖ 多文件编译

示例circle.c circle.h cube.c cube.h test.c编译

```
#include "cube.h"
#include "circle.h"

int main()
{
    cube();
    circle();
    printf("This is a test program\n");
    return 0;
}
```

test.c



Makefile基础

❖ 多文件编译

示例circle.c

circle.h

cube.c

cube.h

test.c编译

```
1  TAR = test
2  CC := gcc
3
4  $(TAR):circle.o cube.o test.o
5      $(CC) circle.o cube.o test.o -o $(TAR)
6  circle.o:circle.c
7      $(CC) -c circle.c -o test.o
8  cube.o:cube.c
9      $(CC) -c cube.c -o cube.o
10 test.o:test.c
11     $(CC) -c test.c -o test.o
12
13 .PHONY:      #伪目标
14 clean:      #make clean调用
15     rm -rf circle.o cube.o test.o test
```



Makefile基础-通配符

通配符	含义
%	模式字符，用来通配任意个字符
\$@	目标文件名。多目标模式规则中，它代表的是触发规则被执行的文件名
\$%	当目标文件是一个静态库文件时，代表静态库的一个成员名
\$<	第一个依赖的文件名
\$?	所有比目标文件更新的依赖文件列表
^	代表的是所有依赖文件列表
+	保留了依赖文件中重复出现的文件。主要用在程序链接时库的交叉引用场合



Makefile基础-通配符

```
1 TAR = test
2 CC := gcc
3
4 $(TAR):circle.o cube.o test.o
5     $(CC) circle.o cube.o test.o -o $(TAR)
6 circle.o:circle.c
7     $(CC) -c circle.c -o test.o
8 cube.o:cube.c
9     $(CC) -c cube.c -o cube.o
10 test.o:test.c
11     $(CC) -c test.c -o test.o
12
13 .PHONY:      #伪目标
14 clean:      #make clean调用
15     rm -rf circle.o cube.o test.o test
```

make clean 清除

```
1 TAR = test
2 OBJ = circle.o cube.o test.o
3 CC := gcc
4
5 $(TAR):$(OBJ)
6     $(CC) $^ -o $@
7 %.o:%.c
8     $(CC) -c $^ -o $@
9
10 .PHONY:      #伪目标
11 clean:      #make clean调用
12     rm -rf $(OBJ) $(TAR)
```

通配符 \$@ \$^



Makefile基础

❖ 条件判断

If ifeq ifneq ifdef ifndef

以endif结尾

复杂条件判断使用elif和else

Example:

```
libs_for_gcc = -lgnu
normal_libs =
ifeq ($(CC), gcc)
    libs=$(libs_for_gcc)
else
    libs=$(normal_libs)
endif
```



Makefile工作

❖ Make命令工作顺序

- ❧ make在当前目录下寻找“Makefile”或“makefile”文件
- ❧ 若找到，查找文件中的第一个目标文件.o
- ❧ 若目标文件不存在，根据依赖关系查找.s文件
- ❧ 若.s文件不存在，根据依赖关系查找.i文件
- ❧ 若.i文件不存在，根据依赖关系查找.c文件
- ❧ 若.c文件一定存在，于是按顺序依次生成.i、.s、.o文件，再去执行



什么是CMake

- ❖ 不同平台编译所使用的Makefile标准、格式不同
- ❖ CMake编写一种与平台无关的文件来指导整个编译流程，根据目标平台生成所需要的Makefile和工程文件
- ❖ CMake具有最小的依赖性，只需要在工作系统上安装编译器即可



Cmake工作的流程

在linux平台下使用CMake生成Makefile并编译的流程如下：

- ❖ 编写CMake配置文件CMakeLists.txt
- ❖ 执行命令 `cmake PATH` 或者 `ccmake PATH` 生成 Makefile（`ccmake` 和 `cmake` 的区别在于前者提供了一个交互式的界面）。其中，`PATH` 是CMakeLists.txt 所在的目录
- ❖ 使用 `make` 命令进行编译



CMake安装

- ❖ Windows可以去cmake官网下载安装包来进行安装
(<https://cmake.org/download/>)
- ❖ Ubuntu系统上可以使用包管理工具apt（或apt-get）进行安装
`sudo apt install cmake`
- ❖ 指定版本的cmake，可以手动下载源码编译/二进制安装包



推荐的CMake学习资料

- ❖ <https://cmake.org/cmake/help/latest/guide/tutorial/index.html>
- ❖ <https://www.zhihu.com/search?type=content&q=cmake>
- ❖ <https://cliutils.gitlab.io/modern-cmake/>
- ❖ https://modern-cmake-cn.github.io/Modern-CMake-zh_CN/



本章小结

- ❖ 理解模块划分原则
- ❖ 写出规范的头文件（包含保护、自洽可编译）
- ❖ 用命令行完成多文件编译与链接
- ❖ 常见链接错误排查与解决



谢谢!



中国科学技术大学

University of Science and Technology of China